Repair ledger — Histor-lite ODIN Importer

Issues found and fixed during the build, in Project Witness's "make claims, attack
claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): Histor-lite ODIN Importer

First Histor-lite ODIN importer build — Move 17 of Project Witness. It takes a
verified ODIN Claim Reasoner v0.1 output and records it as a Histor-lite-style
reasoning case (summary + verbatim
report/trace/claims copies + replay report + checksums), preserving ODIN's reasoning status exactly, and verifies that case
later. It is not connected to Aletheia, Coordinator, or Companion.

Scope held deliberately narrow: one ODIN reasoning output -> one reasoning case
record; structural metadata import only; no Aletheia

import, no Sentinel data, no
natural language, no LLM, no truth claims, no signing, no anchoring. The importer
records structural reasoning metadata only — it never claims any claim is true.

Design decisions worth recording:

- **Contradiction is a valid outcome, not a failure.** Both `CONSISTENT` and `CONTRADICTION_DETECTED` import successfully as `IMPORTED_VERIFIED_ODIN`; only `INVALID_INPUT` is refused (at import and again at verify). A structural
  contradiction is a real reasoning result. This is covered by explicit
  consistent-accepted and contradiction-accepted tests, plus a real end-to-end
  contradiction integration check.
- **ODIN files copied verbatim, never rewritten.** The report, proof trace, and
  claims are copied byte-for-byte from the source run. ODIN output is already
  byte-for-byte canonical, so the verbatim copies also satisfy verify's canonical
  check — preservation and canonical-form are not in tension. A test asserts the
  copies are byte-identical to the source files.

- **Reasoning status preserved exactly.** The summary's `source_reasoning_status` is copied from the report, and verify requires it to match the report copy and to be an importable status. claim_count is cross-checked against BOTH the report copy and the claims copy length; contradiction_count against the report copy; proof_step_count against the trace copy's steps length.
- **Truth-claim guard at verify.** The replay report records `truth_claim: none` plus an explicit disclaimer sentence, and verify scans BOTH the summary and the replay report for forbidden truth/authenticity tokens, refusing with `TRUTH_CLAIM` even after the checksums are resynced.
- **INVALID_INPUT injected after import is still caught.** A dedicated test injects `INVALID_INPUT` into both the summary and the report copy (so the status cross-check would pass) and resyncs the checksums; verify still refuses it with `REASONING_STATUS_INVALID`.
- **Verify recomputes cross-references; it does not trust the summary.** Every count and status is checked against the copied source files after a canonical + hash

check, so resyncing the checksums after a lie does not help.

- **Exactly six files, nothing else.** Verify refuses any unknown extra file.
- **Strict, lowercase-only checksums.** Exactly five keys; duplicate/missing/ unknown keys fail; malformed digests fail; uppercase digests fail.
- **Frozen boundary is injectable.** ODIN verify is called via subprocess at runtime, but the unit suite injects an in-process fake verifier, so the freeze gate spawns no subprocesses and does not require the real frozen ODIN folder. The subprocess path is exercised only by the documented manual smoke test.
- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output directory; `require()` not bare assert; in-process selfcheck that never launches the unit suite.

Clean first build: no behavioural self-fixes were required; the ledger records the design decisions and the confirmed end-to-end integration (both consistent and contradiction) rather than repairs.

Real end-to-end integration confirmed. Against frozen ODIN v0.1: built a real CONSISTENT run -> `import-reasoning` produced `IMPORTED_VERIFIED_ODIN` with `source_reasoning_status = CONSISTENT`, claim/contradiction/proof-step counts preserved, the three copies byte-identical to source, `truth_claim: none` in the replay report; `verify-case` passed. Also built a real CONTRADICTION_DETECTED run (two claims, same predicate, AFFIRMED + DENIED) -> imported as `IMPORTED_VERIFIED_ODIN` with the contradiction status preserved; `verify-case` passed.

Release gate (lesson applied directly): no subprocess orchestrator in the freeze gate. The authoritative gate is the explicit command list in RELEASE_COMMANDS.md plus the static `verify_release_manifest.py` (launches no subprocesses; additionally checks the README scopes the importer to structural reasoning metadata only, disclaims proving truth, says it does not import Aletheia evidence metadata, and says it is not connected to Coordinator or Companion yet).

Verified: all release-gate commands pass individually from a clean extraction

(see TEST_RESULTS.md); selfcheck 8/8 in-process gates; verify_release_manifest.py
passes (10/10, no subprocesses); tamper to any case file, the checksums, the file
set, the cross-referenced ids/statuses/counts, an injected INVALID_INPUT, or an
injected truth claim is detected with the correct reason.